读享 V1.0 软件

后 30 页

著作权人:XX 有限公司

生成日期:2026-06-15

摘选总行数:1500

文件清单:

- person/notfresh/readingshare/core/Synchronizer.java
- person/notfresh/readingshare/util/PdfOutlineExtractor.java
- person/notfresh/readingshare/util/BilibiliUrlConverter.java
- person/notfresh/readingshare/util/CrawlUtil.java
- person/notfresh/readingshare/util/ExportUtil.java
- person/notfresh/readingshare/util/ImportUtil.java

```
  1  package person.notfresh.readingshare.core;
  2
  3  import android.util.Log;
  4  import person.notfresh.readingshare.model.LinkItem;
  5
  6  import org.json.JSONArray;
  7  import org.json.JSONException;
  8  import org.json.JSONObject;
  9
 10  import java.io.BufferedReader;
 11  import java.io.IOException;
 12  import java.io.InputStreamReader;
 13  import java.io.OutputStream;
 14  import java.net.HttpURLConnection;
 15  import java.net.URL;
 16  import java.nio.charset.StandardCharsets;
 17  import java.text.SimpleDateFormat;
 18  import java.util.Date;
 19  import java.util.Iterator;
 20  import java.util.List;
 21  import java.util.Locale;
 22
 23  /**
 24   * ■■■■■■■■■■■■■■■■URL
 25   *
 26   * ■■■■■
 27   * 1. ■■ List<LinkItem> ■ URL
 28   * 2. ■■■■■■■■JSON■■
 29   * 3. ■■POST■■■■■■■URL
 30   * 4. ■■■■■■
 31   */
 32  public class Synchronizer {
 33      private static final String TAG = "Synchronizer";
 34
 35      /**
 36       * ■■■■■■■■■URL
 37       *
 38       * @param links ■■■■■■■■
 39       * @param targetUrl ■■URL
 40       * @return ■■■■
 41       */
 42      public SyncResult synchronize(List<LinkItem> links, String targetUrl) {
 43          return synchronize(links, targetUrl, null);
 44      }
 45
 46      /**
 47       * ■■■■■■■■■URL■■■■■■■
 48       *
 49       * @param links ■■■■■■■■
 50       * @param targetUrl ■■URL
```

```
51       * @param extraData ■■■JSON■■■■■■■■■■■■■■■
52       * @return ■■■■
53       */
54      public SyncResult synchronize(List<LinkItem> links, String targetUrl, JSONOb
55          if (links == null || links.isEmpty()) {
56              return SyncResult.failure("■■■■■■■");
57          }
58
59          if (targetUrl == null || targetUrl.isEmpty()) {
60              return SyncResult.failure("■■URL■■");
61          }
62
63          try {
64              // ■■JSON■■
65              JSONObject jsonData = buildJsonData(links, extraData);
66              Log.d(TAG, "■■■ JSON ■■: " + jsonData.toString());
67
68              // ■■■■■■
69              return sendRequest(targetUrl, jsonData);
70
71          } catch (JSONException e) {
72              Log.e(TAG, "JSON ■■■■", e);
73              return SyncResult.failure("■■■■■■: " + e.getMessage());
74          } catch (Exception e) {
75              Log.e(TAG, "■■■■", e);
76              return SyncResult.failure("■■■■: " + e.getMessage());
77          }
78      }
79
80      /**
81       * ■■JSON■■
82       */
83      private JSONObject buildJsonData(List<LinkItem> links, JSONObject extraData)
84          JSONObject jsonData = new JSONObject();
85
86          // ■■■■■■■■■■■■■
87          if (extraData != null) {
88              Iterator<String> keys = extraData.keys();
89              while (keys.hasNext()) {
90                  String key = keys.next();
91                  jsonData.put(key, extraData.get(key));
92              }
93          }
94
95          // ■■■■■■
96          JSONArray linksArray = new JSONArray();
97          SimpleDateFormat isoFormat = new SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss
98
99          for (LinkItem link : links) {
100             JSONObject linkObj = new JSONObject();
```

```
101             linkObj.put("title", link.getTitle());
102             linkObj.put("url", link.getUrl());
103             linkObj.put("timestamp", isoFormat.format(new Date(link.getTimestamp
104             linksArray.put(linkObj);
105         }
106 
107         jsonData.put("links", linksArray);
108         return jsonData;
109     }
110 
111     /**
112      * ■■HTTP■■
113      */
114     private SyncResult sendRequest(String targetUrl, JSONObject jsonData) {
115         try {
116             URL url = new URL(targetUrl);
117             HttpURLConnection conn = (HttpURLConnection) url.openConnection();
118 
119             // TODO ■■SSL■■■■■■■■■
120             //configureSSL(conn);
121 
122             // ■■■■■■■■■
123             conn.setRequestMethod("POST");
124             conn.setRequestProperty("Content-Type", "application/json");
125             conn.setDoOutput(true);
126 
127             Log.d(TAG, "■■■■■■■: " + targetUrl);
128 
129             // ■■■■■
130             try (OutputStream os = conn.getOutputStream()) {
131                 byte[] input = jsonData.toString().getBytes(StandardCharsets.UTF
132                 os.write(input, 0, input.length);
133             }
134 
135             // ■■■■
136             int responseCode = conn.getResponseCode();
137             Log.d(TAG, "■■■■■■: " + responseCode);
138 
139             if (responseCode == HttpURLConnection.HTTP_OK) {
140                 // ■■■■■■
141                 String responseData = readResponse(conn);
142                 return SyncResult.success(responseData, responseCode);
143             } else {
144                 // ■■■■■■
145                 String errorResponse = readErrorResponse(conn);
146                 return SyncResult.failure("■■■■■■■: " + responseCode +
147                     (errorResponse != null ? " - " + errorResponse : ""), respon
148             }
149 
150         } catch (IOException e) {
```

```
151             Log.e(TAG, "■■■■■■■", e);
152             return SyncResult.failure("■■■■■■■: " + e.getMessage());
153         } catch (Exception e) {
154             Log.e(TAG, "■■■■■■■■", e);
155             return SyncResult.failure("■■■■■■■: " + e.getMessage());
156         }
157     }
158 
159     /**
160      * ■■SSL■■■■■■■■■
161      * ■■■■■■■■■■■■■■■■■■■■■■■■■
162      */
163     private void configureSSL(HttpURLConnection conn) {
164         if (conn instanceof javax.net.ssl.HttpsURLConnection) {
165             try {
166                 javax.net.ssl.HttpsURLConnection httpsConn = (javax.net.ssl.Http
167                 javax.net.ssl.TrustManager[] trustAllCerts = new javax.net.ssl.T
168                     new javax.net.ssl.X509TrustManager() {
169                         public java.security.cert.X509Certificate[] getAcceptedI
170                             return null;
171                         }
172                         public void checkClientTrusted(java.security.cert.X509Ce
173                         }
174                         public void checkServerTrusted(java.security.cert.X509Ce
175                         }
176                     }
177                 };
178 
179                 javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstan
180                 sc.init(null, trustAllCerts, new java.security.SecureRandom());
181                 httpsConn.setSSLSocketFactory(sc.getSocketFactory());
182                 httpsConn.setHostnameVerifier((hostname, session) -> true);
183             } catch (Exception e) {
184                 Log.e(TAG, "■■SSL■■", e);
185             }
186         }
187     }
188 
189     /**
190      * ■■■■■■■
191      */
192     private String readResponse(HttpURLConnection conn) throws IOException {
193         try (BufferedReader reader = new BufferedReader(
194                 new InputStreamReader(conn.getInputStream(), StandardCharsets.UT
195             StringBuilder response = new StringBuilder();
196             String line;
197             while ((line = reader.readLine()) != null) {
198                 response.append(line);
199             }
200             return response.toString();
```

```
201          }
202      }
203
204      /**
205       * ■■■■■■
206       */
207      private String readErrorResponse(HttpURLConnection conn) {
208          try {
209              if (conn.getErrorStream() != null) {
210                  try (BufferedReader reader = new BufferedReader(
211                          new InputStreamReader(conn.getErrorStream(), StandardCha
212                      StringBuilder response = new StringBuilder();
213                      String line;
214                      while ((line = reader.readLine()) != null) {
215                          response.append(line);
216                      }
217                      return response.toString();
218                  }
219              }
220          } catch (IOException e) {
221              Log.e(TAG, "■■■■■■■■", e);
222          }
223          return null;
224      }
225  }
226
```

```
 1  package person.notfresh.readingshare.util;
 2
 3  import android.util.Log;
 4
 5  import java.io.File;
 6  import java.io.FileInputStream;
 7  import java.io.IOException;
 8  import java.nio.charset.StandardCharsets;
 9  import java.util.ArrayList;
10  import java.util.List;
11  import java.util.regex.Matcher;
12  import java.util.regex.Pattern;
13
14  import person.notfresh.readingshare.ui.document.TocOutlineItem;
15
16  /**
17   * PDF■■■■■
18   * ■■■■PDF■■■■■■■■■■/■■■■
19   */
20  public class PdfOutlineExtractor {
21      private static final String TAG = "PdfOutlineExtractor";
22
23      /**
24       * ■PDF■■■■■■■■
25       *
26       * @param pdfFile PDF■■
27       * @return ■■■■■
28       */
29      public static List<TocOutlineItem> extractOutline(File pdfFile) {
30          List<TocOutlineItem> outlineItems = new ArrayList<>();
31
32          if (pdfFile == null || !pdfFile.exists()) {
33              Log.w(TAG, "PDF■■■■■");
34              return outlineItems;
35          }
36
37          try {
38              // ■■PDF■■■■
39              byte[] pdfBytes = readFileBytes(pdfFile);
40              String pdfContent = new String(pdfBytes, StandardCharsets.ISO_8859_1
41
42              // ■■Outlines■■
43              extractOutlinesFromContent(pdfContent, outlineItems);
44
45              Log.d(TAG, "■■■ " + outlineItems.size() + " ■■■■");
46
47          } catch (Exception e) {
48              Log.e(TAG, "■■PDF■■■■", e);
49          }
50
```

```
51          return outlineItems;
52      }
53
54      /**
55       * ■■■■■■■
56       */
57      private static byte[] readFileBytes(File file) throws IOException {
58          FileInputStream fis = new FileInputStream(file);
59          byte[] buffer = new byte[(int) file.length()];
60          fis.read(buffer);
61          fis.close();
62          return buffer;
63      }
64
65      /**
66       * ■PDF■■■■■■■■
67       * PDF■■■■■■■■■■Outlines■■■
68       */
69      private static void extractOutlinesFromContent(String pdfContent, List<TocOu
70          try {
71              // ■■1■■■■■■■■ /Title ■ /Dest ■■■■■■
72              // PDF■■■■■■<< /Title (■■) /Dest [...] /Parent ... /Next ... /First .
73              // ■■■■■■■■■■■■■
74              Pattern bookmarkPattern = Pattern.compile(
75                  "<<[^>]*?/Title\\s*\\(([^)]+)\\)[^>]*?/Dest\\s*\\[([^\\]]+)\\]",
76                  Pattern.CASE_INSENSITIVE | Pattern.DOTALL
77              );
78
79              Matcher bookmarkMatcher = bookmarkPattern.matcher(pdfContent);
80              int count = 0;
81
82              while (bookmarkMatcher.find() && count < 200) { // ■■■■200■
83                  try {
84                      String title = bookmarkMatcher.group(1);
85                      String dest = bookmarkMatcher.group(2);
86
87                      // ■■■■
88                      int pageIndex = extractPageIndexFromDest(dest);
89
90                      if (pageIndex >= 0) {
91                          // ■■■■
92                          title = cleanTitle(title);
93
94                          // ■■■■■■■■■■■■■■■■■
95                          if (isValidOutlineTitle(title)) {
96                              // ■■■■■■■■■■■■Parent/First/Last■■■■■■■■■
97                              int level = estimateLevel(bookmarkMatcher.start(), p
98
99                              TocOutlineItem item = new TocOutlineItem(title, page
100                             outlineItems.add(item);
```

```
101                             count++;
102
103                             Log.d(TAG, "■■■■■: " + title + " -> ■" + (pageIndex
104                         }
105                     }
106                 } catch (Exception e) {
107                     Log.w(TAG, "■■■■■■■■", e);
108                 }
109             }
110
111             // ■■2■■■■■1■■■■■■■■■■ /Title ■■
112             if (outlineItems.isEmpty()) {
113                 extractOutlinesSimple(pdfContent, outlineItems);
114             }
115
116             // ■■■■■
117             deduplicateAndSort(outlineItems);
118
119         } catch (Exception e) {
120             Log.e(TAG, "■■PDF■■■■", e);
121         }
122     }
123
124     /**
125      * ■■■■■■■■■■■■
126      */
127     private static boolean isValidOutlineTitle(String title) {
128         if (title == null || title.trim().isEmpty()) {
129             return false;
130         }
131
132         // ■■■■■■■■■■■
133         String lowerTitle = title.toLowerCase();
134         String[] invalidTitles = {
135             "metadata", "info", "xref", "trailer", "catalog",
136             "root", "pages", "page", "outlines", "acroform"
137         };
138
139         for (String invalid : invalidTitles) {
140             if (lowerTitle.equals(invalid)) {
141                 return false;
142             }
143         }
144
145         // ■■■■■■■■■■■■
146         return title.length() >= 1 && title.length() <= 200;
147     }
148
149     /**
150      * ■■■■■■■■
```

```
151         * ■■■■■■■Parent/First■■■■■
152         */
153        private static int estimateLevel(int position, String pdfContent) {
154            try {
155                // ■■■■■■■■■■Parent■■
156                int start = Math.max(0, position - 1000);
157                int end = Math.min(pdfContent.length(), position + 1000);
158                String context = pdfContent.substring(start, position);
159
160                // ■■Parent■■■■■■■■■Parent■■■■■■■■
161                Pattern parentPattern = Pattern.compile("/Parent");
162                Matcher parentMatcher = parentPattern.matcher(context);
163                int parentCount = 0;
164                while (parentMatcher.find()) {
165                    parentCount++;
166                }
167
168                return Math.min(parentCount, 5); // ■■5■
169            } catch (Exception e) {
170                return 0;
171            }
172        }
173
174        /**
175         * ■■■■■■■■■
176         */
177        private static void deduplicateAndSort(List<TocOutlineItem> items) {
178            if (items.isEmpty()) {
179                return;
180            }
181
182            // ■■■■■■
183            items.sort((a, b) -> Integer.compare(a.getPageIndex(), b.getPageIndex())
184
185            // ■■■■■■■■■■■■■■■■■■■■■■
186            List<TocOutlineItem> unique = new ArrayList<>();
187            TocOutlineItem last = null;
188
189            for (TocOutlineItem item : items) {
190                if (last == null ||
191                    !item.getTitle().equals(last.getTitle()) ||
192                    item.getPageIndex() != last.getPageIndex()) {
193                    unique.add(item);
194                    last = item;
195                }
196            }
197
198            items.clear();
199            items.addAll(unique);
200        }
```

```
201
202     /**
203      * ■Dest■■■■■■■■
204      * Dest■■■■■■
205      * - [3 0 R] -> ■■■3■■■■■■
206      * - [3 /XYZ null null null] -> ■■■3
207      * - [0 3 R] -> ■■■3■■■■■■■■■
208      * - [page 3 /XYZ ...] -> ■■■3
209      * - 3 0 R -> ■■■3■■■■■■
210      */
211     private static int extractPageIndexFromDest(String dest) {
212         if (dest == null || dest.trim().isEmpty()) {
213             return -1;
214         }
215
216         try {
217             // ■■■■■
218             dest = dest.trim().replaceAll("[\\[\\]]", "").trim();
219
220             // ■■1■■■ /XYZ ■ /Fit ■■■■■■■■■■■■
221             Pattern xyzPattern = Pattern.compile("(\\d+)\\s+/XYZ|(\\d+)\\s+/Fit"
222             Matcher xyzMatcher = xyzPattern.matcher(dest);
223             if (xyzMatcher.find()) {
224                 String pageStr = xyzMatcher.group(1) != null ? xyzMatcher.group(
225                 int pageNumber = Integer.parseInt(pageStr);
226                 if (pageNumber > 0 && pageNumber < 10000) {
227                     return Math.max(0, pageNumber - 1);
228                 }
229             }
230
231             // ■■2■■■■■■■■■■■■■■■■■■
232             Pattern pagePattern = Pattern.compile("\\b(\\d{1,4})\\b");
233             Matcher pageMatcher = pagePattern.matcher(dest);
234
235             while (pageMatcher.find()) {
236                 try {
237                     int num = Integer.parseInt(pageMatcher.group(1));
238                     // ■■■■■■■■■■■■■■■0■■■■■■■■
239                     if (num > 0 && num < 10000) {
240                         // PDF■■■1■■■■■■■0-based■■
241                         return Math.max(0, num - 1);
242                     }
243                 } catch (NumberFormatException ignored) {
244                 }
245             }
246
247             // ■■3■■■Dest■■■■■■■■■■ "3 0 R"■■■■■■■■■■
248             Pattern refPattern = Pattern.compile("^(\\d+)\\s+\\d+\\s+R");
249             Matcher refMatcher = refPattern.matcher(dest);
250             if (refMatcher.find()) {
```

```
                int pageNumber = Integer.parseInt(refMatcher.group(1));
                if (pageNumber > 0 && pageNumber < 10000) {
                    return Math.max(0, pageNumber - 1);
                }
            }

        } catch (Exception e) {
            Log.w(TAG, "■■■■■■: " + dest, e);
        }

        return -1;
    }

    /**
     * ■■■■■■
     * ■■PDF■■■■■■■■■■■■■
     */
    private static String cleanTitle(String title) {
        if (title == null) {
            return "";
        }

        // ■■PDF■■■■
        title = title.replace("\\(", "(")
                     .replace("\\)", ")")
                     .replace("\\n", "\n")
                     .replace("\\r", "\r")
                     .replace("\\t", "\t")
                     .replace("\\\\", "\\");

        // ■■■■■■■■
        title = fixEncoding(title);

        // ■■■■■■■
        title = title.trim();

        return title;
    }

    /**
     * ■■■■■■■■■
     * PDF■■■■■■■■■■■■■■UTF-8■UTF-16BE■PDFDocEncoding■
     */
    private static String fixEncoding(String text) {
        if (text == null || text.isEmpty()) {
            return text;
        }

        // ■■■■■■■■■■■■■■■■■■■
        if (text.contains("þÿ") || text.contains("ÿþ")) {
```

```
301             // ■■UTF-16BE■■■PDF■■■■Unicode■■■
302             try {
303                 // ■■BOM■■■þÿ = 0xFEFF■
304                 byte[] bytes = text.getBytes(StandardCharsets.ISO_8859_1);
305 
306                 // ■■■■■UTF-16BE BOM (FE FF)
307                 if (bytes.length >= 2 && bytes[0] == (byte)0xFE && bytes[1] == (
308                     // UTF-16BE with BOM
309                     String decoded = new String(bytes, 2, bytes.length - 2, Stan
310                     if (isValidText(decoded)) {
311                         return decoded;
312                     }
313                 }
314 
315                 // ■■■■■UTF-16LE BOM (FF FE)
316                 if (bytes.length >= 2 && bytes[0] == (byte)0xFF && bytes[1] == (
317                     // UTF-16LE with BOM
318                     String decoded = new String(bytes, 2, bytes.length - 2, Stan
319                     if (isValidText(decoded)) {
320                         return decoded;
321                     }
322                 }
323 
324                 // ■■■■■■UTF-16BE■■■■BOM■
325                 try {
326                     String decoded = new String(bytes, StandardCharsets.UTF_16BE
327                     if (isValidText(decoded) && !decoded.contains("þ") && !decod
328                         return decoded;
329                     }
330                 } catch (Exception ignored) {
331                 }
332 
333                 // ■■UTF-8■■
334                 try {
335                     String decoded = new String(bytes, StandardCharsets.UTF_8);
336                     if (isValidText(decoded) && !decoded.contains("þ") && !decod
337                         return decoded;
338                     }
339                 } catch (Exception ignored) {
340                 }
341 
342             } catch (Exception e) {
343                 Log.w(TAG, "■■■■■■: " + text, e);
344             }
345         }
346 
347         // ■■■■■■■■■■■■■■■■
348         if (hasGarbledText(text)) {
349             try {
350                 byte[] bytes = text.getBytes(StandardCharsets.ISO_8859_1);
```

```
351
352                 // ■■UTF-8
353                 String utf8 = new String(bytes, StandardCharsets.UTF_8);
354                 if (isValidText(utf8) && !hasGarbledText(utf8)) {
355                     return utf8;
356                 }
357
358                 // ■■UTF-16BE
359                 if (bytes.length % 2 == 0) {
360                     String utf16be = new String(bytes, StandardCharsets.UTF_16BE
361                     if (isValidText(utf16be) && !hasGarbledText(utf16be)) {
362                         return utf16be;
363                     }
364                 }
365
366             } catch (Exception e) {
367                 Log.w(TAG, "■■■■■■", e);
368             }
369         }
370
371         return text;
372     }
373
374     /**
375      * ■■■■■■■■■■■■
376      */
377     private static boolean hasGarbledText(String text) {
378         if (text == null) {
379             return false;
380         }
381         // ■■■■■■■■■■
382         return text.contains("þ") || text.contains("ÿ") ||
383                text.matches(".*[\\u0000-\\u001F].*") && text.length() > 10;
384     }
385
386     /**
387      * ■■■■■■■■■■■■■■■■■■■■
388      */
389     private static boolean isValidText(String text) {
390         if (text == null || text.isEmpty()) {
391             return false;
392         }
393
394         // ■■■■■■■■■■■■■■■
395         int controlCharCount = 0;
396         for (char c : text.toCharArray()) {
397             if (Character.isISOControl(c) && c != '\n' && c != '\r' && c != '\t'
398                 controlCharCount++;
399             }
400         }
```

```
401
402         // ■■■■■■■■■■■■■■10%■■■■■■■■■■
403         return controlCharCount < text.length() * 0.1;
404     }
405
406     /**
407      * ■■■■■■■■■■■■■■■■
408      * ■■■■■■■ /Title ■■■
409      */
410     private static void extractOutlinesSimple(String pdfContent, List<TocOutline
411         try {
412             // ■■■■■ /Title ■■
413             Pattern titlePattern = Pattern.compile("/Title\\s*\\(([^)]+)\\)");
414             Matcher titleMatcher = titlePattern.matcher(pdfContent);
415
416             int count = 0;
417             while (titleMatcher.find() && count < 100) { // ■■■■■100■
418                 try {
419                     String title = cleanTitle(titleMatcher.group(1));
420
421                     // ■■■■■■■■■■■■■■■■■■■■■■
422                     if (title.length() < 2 || title.length() > 200) {
423                         continue;
424                     }
425
426                     // ■■■■■■■■■■
427                     int start = Math.max(0, titleMatcher.start() - 500);
428                     int end = Math.min(pdfContent.length(), titleMatcher.end() +
429                     String context = pdfContent.substring(start, end);
430
431                     int pageIndex = findPageInContext(context);
432
433                     if (pageIndex >= 0) {
434                         TocOutlineItem item = new TocOutlineItem(title, pageInde
435                         outlineItems.add(item);
436                         count++;
437                     }
438                 } catch (Exception e) {
439                     Log.w(TAG, "■■■■■■■", e);
440                 }
441             }
442
443         } catch (Exception e) {
444             Log.e(TAG, "■■■■■■■■■", e);
445         }
446     }
447
448     /**
449      * ■■■■■■■■■■
450      */
```

```
451      private static int findPageInContext(String context) {
452          // ■■ /Dest [■■ ■ /Page ■■
453          Pattern destPattern = Pattern.compile("/Dest\\s*\\[\\s*(\\d+)");
454          Matcher destMatcher = destPattern.matcher(context);
455          if (destMatcher.find()) {
456              try {
457                  int page = Integer.parseInt(destMatcher.group(1));
458                  return Math.max(0, page - 1);
459              } catch (NumberFormatException ignored) {
460              }
461          }
462
463          // ■■ /Page (■■)
464          Pattern pagePattern = Pattern.compile("/Page\\s*\\(?(\\d+)");
465          Matcher pageMatcher = pagePattern.matcher(context);
466          if (pageMatcher.find()) {
467              try {
468                  int page = Integer.parseInt(pageMatcher.group(1));
469                  return Math.max(0, page - 1);
470              } catch (NumberFormatException ignored) {
471              }
472          }
473
474          return -1;
475      }
476  }
477
```

```
 1  package person.notfresh.readingshare.util;
 2
 3  import java.io.IOException;
 4  import java.net.HttpURLConnection;
 5  import java.net.URL;
 6
 7  public class BilibiliUrlConverter {
 8
 9      public static String getRedirectedUrl(String shortUrl) throws IOException {
10          HttpURLConnection connection = (HttpURLConnection) new URL(shortUrl).ope
11          connection.setInstanceFollowRedirects(false);
12          connection.connect();
13          String redirectedUrl = connection.getHeaderField("Location");
14          connection.disconnect();
15          return redirectedUrl;
16      }
17
18      public static String convertToBilibiliScheme(String url) {
19          if (url == null) {
20              return null;
21          }
22          if (url.startsWith("https://www.bilibili.com/video/")) {
23              return url.replace("https://www.bilibili.com/video/", "bilibili://vi
24          }
25          return url;
26      }
27  }
```

```
 1  package person.notfresh.readingshare.util;
 2
 3  import android.util.Log;
 4  import android.widget.EditText;
 5
 6  import java.io.BufferedReader;
 7  import java.io.IOException;
 8  import java.io.InputStream;
 9
10  import org.json.JSONException;
11  import org.json.JSONObject;
12
13  import android.graphics.Bitmap;
14  import android.graphics.BitmapFactory;
15  import org.jsoup.Jsoup;
16  import org.jsoup.nodes.Document;
17  import org.jsoup.nodes.Element;
18  import org.jsoup.select.Elements;
19  import javax.net.ssl.SSLContext;
20  import javax.net.ssl.TrustManager;
21  import javax.net.ssl.X509TrustManager;
22
23  import java.io.InputStreamReader;
24  import java.net.HttpURLConnection;
25  import java.net.URL;
26  import java.net.URLConnection;
27  import java.nio.charset.StandardCharsets;
28  import java.security.KeyManagementException;
29  import java.security.NoSuchAlgorithmException;
30  import java.security.SecureRandom;
31  import java.security.cert.X509Certificate;
32  import java.util.regex.Matcher;
33  import java.util.regex.Pattern;
34
35  import javax.net.ssl.HttpsURLConnection;
36
37  import person.notfresh.readingshare.config.Config;
38
39  public class CrawlUtil {
40
41      public static String getWeixinArticleTitle(String url) throws IOException {
42          // ■■jsoup■■■■■■■■■■■■■■■■■■■■■■■■
43          Document doc = Jsoup.connect(url)
44                  .userAgent(Config.UserAgent.CHROME_USER_AGENT)
45                  .header("Accept", Config.Headers.ACCEPT)
46                  .header("Accept-Language", Config.Headers.ACCEPT_LANGUAGE)
47                  .header("Accept-Encoding", Config.Headers.ACCEPT_ENCODING)
48                  .header("Connection", Config.Headers.CONNECTION)
49                  .header("Cache-Control", Config.Headers.CACHE_CONTROL)
50                  .timeout(Config.Network.WECHAT_CRAWL_TIMEOUT)
```

```
51                  .maxBodySize(Config.Misc.MAX_BODY_SIZE)
52                  .followRedirects(Config.Misc.FOLLOW_REDIRECTS)
53                  .get();
54
55          // ■■■■■■■■■■■■■■
56          String title = doc.title();  // ■■■■■■■■■■
57          if (title.isEmpty()) {
58              // ■■■■■■■■■■■■■■■■■■■
59              title = doc.select(Config.WeChatSelectors.TITLE_SELECTOR).text();
60          }
61          if (title.isEmpty()) {
62              // ■■■■■■■■■■■■■■■■■■
63              title = doc.select(Config.WeChatSelectors.ACTIVITY_NAME_SELECTOR).te
64          }
65
66          // ■■■■■■■■■■■■■■■■■■■■■■
67          return title;
68      }
69
70      public static String getArticleByUrl(String url) throws IOException {
71          // ■■jsoup■■■■■■■■■■■■■■■■■■■■■■■■
72          Document doc = Jsoup.connect(url)
73                  .userAgent(Config.UserAgent.CHROME_USER_AGENT)
74                  .header("Accept", Config.Headers.ACCEPT)
75                  .header("Accept-Language", Config.Headers.ACCEPT_LANGUAGE)
76                  .header("Accept-Encoding", Config.Headers.ACCEPT_ENCODING)
77                  .header("Connection", Config.Headers.CONNECTION)
78                  .header("Cache-Control", Config.Headers.CACHE_CONTROL)
79                  .timeout(Config.Network.WECHAT_CRAWL_TIMEOUT)
80                  .maxBodySize(Config.Misc.MAX_BODY_SIZE)
81                  .followRedirects(Config.Misc.FOLLOW_REDIRECTS)
82                  .get();
83          // ■■■■■■■
84          StringBuilder contentBuilder = new StringBuilder();
85          Log.d("WebViewActivityMenu", "@1.1");
86          // ■■■■■■■■■■■
87          if (url.contains("juejin.cn")) {
88              // ■■■■■
89              String title = doc.select("h2[data-id], h3[data-id], h4[data-id], h5
90              if (!title.isEmpty()) {
91                  contentBuilder.append("Title: ").append(title).append("\n\n");
92              }
93
94              // ■■■■■■■■■
95              org.jsoup.select.Elements contentElements = doc.select("div.article-
96              for (org.jsoup.nodes.Element element : contentElements) {
97                  String text = element.text().trim();
98                  if (!text.isEmpty()) {
99                      if (element.tagName().startsWith("h")) {
100                         contentBuilder.append("\n").append(text).append("\n");
```

```
101                     } else if (element.tagName().equals("li")) {
102                         contentBuilder.append("- ").append(text).append("\n");
103                     } else {
104                         contentBuilder.append(text).append("\n");
105                     }
106                 }
107             }
108             Log.d("WebViewActivityMenu", "@1.3");
109         } else {
110             // ■■■■■■■■■■■■■■■■■■■
111             String title = doc.title();
112             if (!title.isEmpty()) {
113                 contentBuilder.append("Title: ").append(title).append("\n\n");
114             }
115 
116             // ■■■■■■■
117             org.jsoup.select.Elements paragraphs = doc.select("p");
118             for (org.jsoup.nodes.Element p : paragraphs) {
119                 String text = p.text().trim();
120                 if (!text.isEmpty() && text.length() > 10) {
121                     contentBuilder.append(text).append("\n");
122                 }
123             }
124         }
125 
126         String webContent = contentBuilder.toString().trim();
127 
128         // ■■■■■■■■■■■■■■
129         if (webContent.isEmpty()) {
130             webContent = doc.title();
131         }
132 
133         return webContent;
134     }
135 
136     public static String getUrlSummary(String url, int timeoutSeconds) throws IO
137         // ■■API■■URL
138         String apiUrl = Config.API.DUXIANG_API_URL;
139 
140         // ■■■■
141         URL apiUrlObj = new URL(apiUrl);
142         HttpURLConnection conn = (HttpURLConnection) apiUrlObj.openConnection();
143 
144         // ■■■■■■■■■■■■
145         if (conn instanceof javax.net.ssl.HttpsURLConnection) {
146             javax.net.ssl.HttpsURLConnection httpsConn = (javax.net.ssl.HttpsURL
147             javax.net.ssl.TrustManager[] trustAllCerts = new javax.net.ssl.Trust
148                 new javax.net.ssl.X509TrustManager() {
149                     public java.security.cert.X509Certificate[] getAcceptedIssue
150                         return null;
```

```
151                     }
152                     public void checkClientTrusted(java.security.cert.X509Certif
153                     }
154                     public void checkServerTrusted(java.security.cert.X509Certif
155                     }
156                 }
157             };
158 
159             javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("
160             sc.init(null, trustAllCerts, new java.security.SecureRandom());
161             httpsConn.setSSLSocketFactory(sc.getSocketFactory());
162             httpsConn.setHostnameVerifier((hostname, session) -> true);
163         }
164         conn.setRequestMethod("POST");
165         conn.setConnectTimeout(Config.Network.CONNECT_TIMEOUT);
166         conn.setReadTimeout(timeoutSeconds * 1000);
167         conn.setDoOutput(true);
168         conn.setRequestProperty("Content-Type", Config.Headers.CONTENT_TYPE_JSON
169 
170         // ■■■■JSON
171         JSONObject requestJson = new JSONObject();
172         requestJson.put("url", url);
173         requestJson.put("key", Config.API.DUXIANG_API_KEY);
174 
175         // ■■■■■
176         try (java.io.OutputStream os = conn.getOutputStream()) {
177             byte[] input = requestJson.toString().getBytes("utf-8");
178             os.write(input, 0, input.length);
179         }
180 
181         // ■■■■
182         StringBuilder response = new StringBuilder();
183         try (BufferedReader reader = new BufferedReader(
184             new InputStreamReader(conn.getInputStream(), StandardCharsets.UTF_8)
185             String line;
186             while ((line = reader.readLine()) != null) {
187                 response.append(line);
188             }
189         }
190 
191         // ■■JSON■■
192         JSONObject jsonResponse = new JSONObject(response.toString());
193         if (jsonResponse.getBoolean("success")) {
194             JSONObject data = jsonResponse.getJSONObject("data");
195             return data.getString("summary");
196         }
197 
198         throw new IOException("Failed to get summary from API");
199     }
200 
```

```
201
202     public static String fetchTitleCommon(String url) throws IOException {
203         String title = "";
204         URL urlObj = new URL(url);
205         URLConnection conn = urlObj.openConnection();
206         conn.setConnectTimeout(3000);
207         conn.setReadTimeout(3000);
208         conn.setRequestProperty("User-Agent",
209                 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (K
210
211         BufferedReader reader = new BufferedReader(
212                 new InputStreamReader(conn.getInputStream(), StandardCharsets.UT
213         );
214
215         StringBuilder html = new StringBuilder();
216         String line;
217         int linesRead = 0;
218         while ((line = reader.readLine()) != null && linesRead < 100) {
219             html.append(line);
220             linesRead++;
221         }
222         reader.close();
223
224         Pattern pattern = Pattern.compile("<title>(.*?)</title>", Pattern.CASE_I
225         Matcher matcher = pattern.matcher(html.toString());
226         if (matcher.find()) {
227             title = matcher.group(1).trim();
228             // ■■■■■■■20■■■■■■■■20■■■■■■■■■■
229             if (title.length() > 20) {
230                 title = title.substring(0, 20) + "...";
231             }
232         }
233         return title;
234     }
235
236     /**
237      * ■■■■■■■■ favicon URL
238      * ■■■■■■■■■■■■ HTML ■ head ■■■■■■ </head> ■■■■
239      * @param url ■■ URL
240      * @return favicon URL■■■■■■■■■■ null
241      */
242     public static String getFaviconUrl(String url) throws IOException {
243         try {
244             URL urlObj = new URL(url);
245             HttpURLConnection conn = (HttpURLConnection) urlObj.openConnection()
246
247             // ■■■■■■■■■■ fetchTitleCommon■
248             conn.setConnectTimeout(3000);
249             conn.setReadTimeout(5000);
250             conn.setRequestProperty("User-Agent", Config.UserAgent.CHROME_USER_A
```

```
251             conn.setRequestProperty("Accept", "text/html,application/xhtml+xml,a
252
253             // ■■■■ HTML■■■■■ </head> ■■
254             StringBuilder html = new StringBuilder();
255             try (BufferedReader reader = new BufferedReader(
256                     new InputStreamReader(conn.getInputStream(), StandardCharset
257                 String line;
258                 int linesRead = 0;
259                 final int MAX_LINES = 500;  // ■■■■ 500 ■
260                 final int MAX_SIZE = 50 * 1024;  // ■■ 50KB
261
262                 while ((line = reader.readLine()) != null && linesRead < MAX_LIN
263                     html.append(line).append("\n");
264                     linesRead++;
265
266                     // ■■■■■■ </head> ■■■■■■■■■■
267                     String htmlStr = html.toString().toLowerCase();
268                     if (htmlStr.contains("</head>")) {
269                         Log.d("CrawlUtil", "Found </head> tag, stopping at line
270                         break;
271                     }
272
273                     // ■■■■■■■■
274                     if (html.length() > MAX_SIZE) {
275                         Log.d("CrawlUtil", "Reached max size limit, stopping");
276                         break;
277                     }
278                 }
279             }
280
281             // ■■ Jsoup ■■ HTML ■■■■■■ head ■■■
282             Document doc = Jsoup.parse(html.toString());
283
284             // ■■■■■■ favicon ■■
285             String faviconUrl = null;
286
287             // 1. ■■ <link rel="icon">
288             Elements iconLinks = doc.select("link[rel=icon]");
289             if (!iconLinks.isEmpty()) {
290                 faviconUrl = iconLinks.first().attr("href");
291                 Log.d("CrawlUtil", "Found favicon via rel=icon: " + faviconUrl);
292             }
293
294             // 2. ■■■■■■■■ <link rel="shortcut icon">
295             if (faviconUrl == null || faviconUrl.isEmpty()) {
296                 Elements shortcutIconLinks = doc.select("link[rel=shortcut icon]
297                 if (!shortcutIconLinks.isEmpty()) {
298                     faviconUrl = shortcutIconLinks.first().attr("href");
299                     Log.d("CrawlUtil", "Found favicon via rel=shortcut icon: " +
300                 }
```

```
301             }
302
303             // 3. ■■■■■■■■■ apple-touch-icon■■■■■■■■ favicon■
304             if (faviconUrl == null || faviconUrl.isEmpty()) {
305                 Elements appleIconLinks = doc.select("link[rel=apple-touch-icon]
306                 if (!appleIconLinks.isEmpty()) {
307                     faviconUrl = appleIconLinks.first().attr("href");
308                     Log.d("CrawlUtil", "Found favicon via rel=apple-touch-icon:
309                 }
310             }
311
312             // ■■■ HTML ■■■■ favicon URL■■■■■■■■ URL
313             if (faviconUrl != null && !faviconUrl.isEmpty()) {
314                 faviconUrl = resolveUrl(url, faviconUrl);
315                 Log.d("CrawlUtil", "Resolved favicon URL: " + faviconUrl);
316                 return faviconUrl;
317             }
318
319             // ■■ HTML ■■■■■■■■■■■ /favicon.ico
320             String baseUrl = urlObj.getProtocol() + "://" + urlObj.getHost();
321             if (urlObj.getPort() != -1 && urlObj.getPort() != 80 && urlObj.getPo
322                 baseUrl += ":" + urlObj.getPort();
323             }
324             String defaultFavicon = baseUrl + "/favicon.ico";
325             Log.d("CrawlUtil", "No favicon found in HTML, trying default: " + de
326
327             // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
328             return defaultFavicon;
329
330         } catch (Exception e) {
331             Log.e("CrawlUtil", "Error getting favicon URL: " + e.getMessage(), e
332             // ■■■■■■■■■■ favicon.ico ■■
333             try {
334                 URL urlObj = new URL(url);
335                 String baseUrl = urlObj.getProtocol() + "://" + urlObj.getHost()
336                 if (urlObj.getPort() != -1 && urlObj.getPort() != 80 && urlObj.g
337                     baseUrl += ":" + urlObj.getPort();
338                 }
339                 return baseUrl + "/favicon.ico";
340             } catch (Exception ex) {
341                 Log.e("CrawlUtil", "Error constructing default favicon URL", ex)
342                 return null;
343             }
344         }
345     }
346
347     /**
348      * ■■■ URL ■■■■■ URL
349      * @param baseUrl ■■ URL
350      * @param relativeUrl ■■ URL
```

```
351       * @return ■■ URL
352       */
353      private static String resolveUrl(String baseUrl, String relativeUrl) {
354          if (relativeUrl == null || relativeUrl.isEmpty()) {
355              return null;
356          }
357
358          try {
359              // ■■■■■■■■ URL■■■■■■
360              if (relativeUrl.startsWith("http://") || relativeUrl.startsWith("htt
361                  return relativeUrl;
362              }
363
364              URL base = new URL(baseUrl);
365
366              // ■■■■■■■ URL■//example.com/favicon.ico
367              if (relativeUrl.startsWith("//")) {
368                  return base.getProtocol() + ":" + relativeUrl;
369              }
370
371              // ■■■■■■■■/favicon.ico
372              if (relativeUrl.startsWith("/")) {
373                  return base.getProtocol() + "://" + base.getHost() +
374                         (base.getPort() != -1 && base.getPort() != 80 && base.get
375                         relativeUrl;
376              }
377
378              // ■■■■■■■■favicon.ico ■ ../favicon.ico
379              URL resolved = new URL(base, relativeUrl);
380              return resolved.toString();
381
382          } catch (Exception e) {
383              Log.e("CrawlUtil", "Error resolving URL: " + e.getMessage(), e);
384              // ■■■■■■■■■■■■■■
385              if (relativeUrl.startsWith("/")) {
386                  try {
387                      URL base = new URL(baseUrl);
388                      return base.getProtocol() + "://" + base.getHost() + relativ
389                  } catch (Exception ex) {
390                      return null;
391                  }
392              }
393              return null;
394          }
395      }
396
397      /**
398       * ■■ favicon ■■■■■■■ Bitmap
399       * @param faviconUrl favicon ■■ URL
400       * @return Bitmap ■■■■■■■■■■■■ null
```

```
401       */
402      public static Bitmap downloadFavicon(String faviconUrl) {
403          if (faviconUrl == null || faviconUrl.isEmpty()) {
404              Log.w("CrawlUtil", "Favicon URL is null or empty");
405              return null;
406          }
407
408          try {
409              URL url = new URL(faviconUrl);
410              HttpURLConnection conn = (HttpURLConnection) url.openConnection();
411
412              // ■■■■■■■
413              conn.setConnectTimeout(3000);
414              conn.setReadTimeout(5000);
415              conn.setRequestProperty("User-Agent", Config.UserAgent.CHROME_USER_A
416
417              // ■■■■■
418              try (InputStream inputStream = conn.getInputStream()) {
419                  Bitmap bitmap = BitmapFactory.decodeStream(inputStream);
420
421                  if (bitmap != null) {
422                      Log.d("CrawlUtil", "Successfully downloaded favicon, size: "
423                              bitmap.getWidth() + "x" + bitmap.getHeight());
424
425                      // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
426                      if (bitmap.getWidth() > 256 || bitmap.getHeight() > 256) {
427                          int newSize = Math.min(256, Math.max(bitmap.getWidth(),
428                          float scale = (float) newSize / Math.max(bitmap.getWidth
429                          int newWidth = (int) (bitmap.getWidth() * scale);
430                          int newHeight = (int) (bitmap.getHeight() * scale);
431                          Bitmap scaledBitmap = Bitmap.createScaledBitmap(bitmap,
432                          bitmap.recycle(); // ■■■■■■■
433                          bitmap = scaledBitmap;
434                          Log.d("CrawlUtil", "Scaled favicon to: " + newWidth + "x
435                      }
436
437                      return bitmap;
438                  } else {
439                      Log.w("CrawlUtil", "Failed to decode favicon bitmap");
440                      return null;
441                  }
442              }
443
444          } catch (Exception e) {
445              Log.e("CrawlUtil", "Error downloading favicon from " + faviconUrl +
446              return null;
447          }
448      }
449  }
```

```
  1  package person.notfresh.readingshare.util;
  2
  3  import android.content.ContentResolver;
  4  import android.content.ContentValues;
  5  import android.content.Context;
  6  import android.net.Uri;
  7  import android.os.Build;
  8  import android.os.Environment;
  9  import android.provider.MediaStore;
 10  import android.text.TextUtils;
 11  import org.json.JSONArray;
 12  import org.json.JSONObject;
 13  import java.io.File;
 14  import java.io.FileOutputStream;
 15  import java.io.IOException;
 16  import java.io.OutputStream;
 17  import java.io.OutputStreamWriter;
 18  import java.nio.charset.StandardCharsets;
 19  import java.text.SimpleDateFormat;
 20  import java.util.Date;
 21  import java.util.List;
 22  import java.util.Locale;
 23  import person.notfresh.readingshare.model.LinkItem;
 24  import android.util.Log;
 25  import org.json.JSONException;
 26
 27  public class ExportUtil {
 28
 29      /**
 30       * ■■■JSON■■■■■■■■■■■■
 31       */
 32      public static String exportToJson(Context context, List<LinkItem> links, Str
 33          JSONArray jsonArray = new JSONArray();
 34
 35          for (LinkItem link : links) {
 36              try {
 37                  JSONObject jsonObject = new JSONObject();
 38                  jsonObject.put("title", link.getTitle());
 39                  jsonObject.put("url", link.getUrl());
 40                  jsonObject.put("tags", new JSONArray(link.getTags()));
 41                  jsonArray.put(jsonObject);
 42              } catch (JSONException e) {
 43                  Log.e("ExportUtil", "Error creating JSON object", e);
 44              }
 45          }
 46
 47          // ■■■■■■.json■■
 48          if (!fileName.toLowerCase().endsWith(".json")) {
 49              fileName += ".json";
 50          }
```

```
51
52          File exportDir = new File(context.getExternalFilesDir(null), "exports");
53          if (!exportDir.exists()) {
54              exportDir.mkdirs();
55          }
56
57          File file = new File(exportDir, fileName);
58          // ■■ UTF-8 ■■■■■■
59          OutputStreamWriter writer = new OutputStreamWriter(
60              new FileOutputStream(file), StandardCharsets.UTF_8);
61          // ■ JSONObject ■■■■■■■■■\/ -> /■■■ JSON ■■■
62          String jsonString = jsonArray.toString(4).replace("\\/", "/");
63          writer.write(jsonString);
64          writer.flush();
65          writer.close();
66
67          return file.getAbsolutePath();
68      }
69
70      /**
71       * ■■■CSV■■■■■■■■■■■
72       */
73      public static String exportToCsv(Context context, List<LinkItem> links, Stri
74          // ■■■■■■.csv■■
75          if (!fileName.toLowerCase().endsWith(".csv")) {
76              fileName += ".csv";
77          }
78
79          File exportDir = new File(context.getExternalFilesDir(null), "exports");
80          if (!exportDir.exists()) {
81              exportDir.mkdirs();
82          }
83
84          File file = new File(exportDir, fileName);
85          // ■■ UTF-8 ■■■■■■
86          OutputStreamWriter writer = new OutputStreamWriter(
87              new FileOutputStream(file), StandardCharsets.UTF_8);
88
89          // ■■CSV■■
90          StringBuilder csv = new StringBuilder();
91          // ■■CSV■■■
92          csv.append("■■,■■,■■,■■,■■■■,■■\n");
93
94          SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Local
95
96          // ■■■■■■■
97          for (LinkItem link : links) {
98              String title = escapeCSV(link.getTitle());
99              String url = escapeCSV(link.getUrl());
100             String date = sdf.format(new Date(link.getTimestamp()));
```

```
101             String tags = escapeCSV(TextUtils.join(",", link.getTags()));
102             String clickCount = String.valueOf(link.getClickCount());
103             String summary = escapeCSV(link.getSummary());
104
105             csv.append(String.format("%s,%s,%s,%s,%s,%s\n",
106                     title, url, date, tags, clickCount, summary));
107         }
108
109         writer.write(csv.toString());
110         writer.flush();
111         writer.close();
112
113         return file.getAbsolutePath();
114     }
115
116     /**
117      * ■■■■■■■JSON■■■■■■■■■
118      */
119     public static String exportToJson(Context context, List<LinkItem> links) thr
120         // ■■■■■■■■
121         String fileName = "links_" + getCurrentTime() + "_readshare.json";
122         // ■■■■■■
123         return exportToJson(context, links, fileName);
124     }
125
126     /**
127      * ■■■■■■■CSV■■■■■■■■■
128      */
129     public static String exportToCsv(Context context, List<LinkItem> links) thro
130         // ■■■■■■■■
131         String fileName = "links_" + getCurrentTime() + "_readshare.csv";
132         // ■■■■■■
133         return exportToCsv(context, links, fileName);
134     }
135
136     /**
137      * ■■■■■■■■■■■■■■■■■■
138      * @return ■■■■■■■■■■■■ "20250105_143022"
139      */
140     public static String getCurrentTime() {
141         return new SimpleDateFormat("yyyyMMdd_HHmmss", Locale.getDefault())
142                 .format(new Date());
143     }
144
145     /**
146      * ■■■■■■ Documents ■■■Android 10+ ■■ MediaStore■■■■■■■■■■■■
147      * @param context Context
148      * @param links ■■■■■■■■■
149      * @param isJson ■■■■ JSON ■■■false ■ CSV■
150      * @return ■■■■■■ URI
```

```
151       * @throws IOException ■■■■■■
152       * @throws JSONException JSON ■■■■
153       */
154      public static Uri exportToPublicDirectory(Context context, List<LinkItem> li
155                                                boolean isJson) throws IOException,
156          // ■■■■■■■■■■■■■■■■
157          String defaultFileName = isJson
158              ? "links_" + getCurrentTime() + "_readshare.json.txt"
159              : "links_" + getCurrentTime() + "_readshare.csv";
160          return exportToPublicDirectory(context, links, isJson, defaultFileName);
161      }
162
163      /**
164       * ■■■■■ Documents ■■■■■■■■■■■■
165       * ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
166       * @param context Context
167       * @param links ■■■■■■■■■
168       * @param isJson ■■■ JSON ■■■false ■ CSV■
169       * @param fileName ■■■■■■■■■■■■■■■■
170       * @return ■■■■■■ URI
171       * @throws IOException ■■■■■■■
172       * @throws JSONException JSON ■■■■
173       */
174      public static Uri exportToPublicDirectory(Context context, List<LinkItem> li
175                                                boolean isJson, String fileName) th
176          String mimeType = isJson ? "text/plain" : "text/csv";
177
178          if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q) {
179              // Android 10+ ■■ MediaStore API
180              ContentValues values = new ContentValues();
181              values.put(MediaStore.MediaColumns.DISPLAY_NAME, fileName);
182              values.put(MediaStore.MediaColumns.MIME_TYPE, mimeType);
183              values.put(MediaStore.MediaColumns.RELATIVE_PATH, Environment.DIRECT
184
185              ContentResolver resolver = context.getContentResolver();
186              Uri fileUri = resolver.insert(MediaStore.Files.getContentUri("extern
187
188              if (fileUri != null) {
189                  OutputStream outputStream = resolver.openOutputStream(fileUri);
190                  if (outputStream != null) {
191                      writeDataToStream(outputStream, links, isJson);
192                      outputStream.close();
193                      return fileUri;
194                  }
195              }
196              throw new IOException("■■■■■■■");
197          } else {
198              // Android 9 ■■■■■■■■■■■■
199              File documentsFolder = Environment.getExternalStoragePublicDirectory
200                  Environment.DIRECTORY_DOCUMENTS);
```

```
201             if (!documentsFolder.exists()) {
202                 documentsFolder.mkdirs();
203             }
204 
205             File outputFile = new File(documentsFolder, fileName);
206             FileOutputStream fos = new FileOutputStream(outputFile);
207             writeDataToStream(fos, links, isJson);
208             fos.close();
209 
210             return Uri.fromFile(outputFile);
211         }
212     }
213 
214     /**
215      * ■■■■■■■■
216      * @param outputStream ■■■
217      * @param links ■■■■
218      * @param isJson ■■■ JSON ■■
219      * @throws IOException ■■■■■■
220      * @throws JSONException JSON ■■■■
221      */
222     private static void writeDataToStream(OutputStream outputStream, List<LinkIt
223                                           boolean isJson) throws IOException, JSO
224         OutputStreamWriter writer = new OutputStreamWriter(
225             outputStream, StandardCharsets.UTF_8);
226 
227         if (isJson) {
228             // ■■ JSON ■■
229             JSONArray jsonArray = new JSONArray();
230             for (LinkItem link : links) {
231                 try {
232                     JSONObject jsonObject = new JSONObject();
233                     jsonObject.put("title", link.getTitle());
234                     jsonObject.put("url", link.getUrl());
235                     jsonObject.put("tags", new JSONArray(link.getTags()));
236                     jsonArray.put(jsonObject);
237                 } catch (JSONException e) {
238                     Log.e("ExportUtil", "Error creating JSON object", e);
239                 }
240             }
241             // ■ JSONObject ■■■■■■■■■\/ -> /■■■ JSON ■■■
242             String jsonString = jsonArray.toString(4).replace("\\/", "/");
243             writer.write(jsonString);
244         } else {
245             // ■■ CSV ■■
246             StringBuilder csv = new StringBuilder();
247             csv.append("■■,■■,■■,■■,■■■■,■■\n");
248 
249             SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss", L
250 
```